

Students with 30% reduction can omit: Point 2 in Exercise B (Ranking), Scenario 2 in Exercise C (Physical databases) and Point c of Exercise D (Triggers).

A. Concurrency Control (9 points)

Present a schedule S consisting of as few write operations as possible and no read operation such that the schedule is a member of 2PL and TS' (i.e., TS with Thomas Rule), but not a member of Strict 2PL, TS Mono and TS Multi.

1. Justify memberships and non-memberships of S .
2. Argue why no smaller schedule would have the same (non-)memberships.

NB For TS-Multi, adopt the convention according to which the request $w_{ts}(x)$ is rejected if $ts < \text{WTM}_N(x)$.

B. Ranking (6 points)

A group of journalists is trying to determine the best male tennis player of the Open Era (i.e., since 1968). They base their analysis on the following tables, reporting wins in the four Grand Slam tournaments: Australian Open (AO), Roland-Garros French Open (RG), US Open (US), and Wimbledon (WI). Clearly, more wins are better than fewer wins.

Table 1 for Point 1.

	<i>AO</i>	<i>RG</i>	<i>US</i>	<i>WI</i>
Agassi:	4	1	2	1
Becker:	2	0	1	3
Borg:	0	6	0	5
Connors:	1	0	5	0
Djokovic:	10	2	3	7
Edberg:	2	0	2	2
Federer:	6	1	5	8
Lendl:	2	3	3	0
McEnroe:	0	0	4	3
Nadal:	2	14	4	2
Sampras:	2	0	5	7
Wilander:	3	3	1	0

Table 2 for Point 2.

<i>AO</i>		<i>RG</i>		<i>US</i>		<i>WI</i>	
Djokovic:	10	Nadal:	14	Sampras:	5	Federer:	8
Federer:	6	Borg:	6	Federer:	5	Sampras:	7
Agassi:	4	Wilander:	3	Connors:	5	Djokovic:	7
Wilander:	3	Lendl:	3	Nadal:	4	Borg:	5
Sampras:	2	Djokovic:	2	McEnroe:	4	McEnroe:	3
Nadal:	2	Federer:	1	Lendl:	3	Becker:	3
Lendl:	2	Agassi:	1	Djokovic:	3	Nadal:	2
Edberg:	2	Sampras:	0	Edberg:	2	Edberg:	2
Becker:	2	McEnroe:	0	Agassi:	2	Agassi:	1
Connors:	1	Edberg:	0	Wilander:	1	Wilander:	0
McEnroe:	0	Connors:	0	Becker:	1	Lendl:	0
Borg:	0	Becker:	0	Borg:	0	Connors:	0

1. In order to provide a good starting point for the journalists, please identify the skyline of the dataset. Just indicate the elements of the skyline; no intermediate steps need to be specified.
2. Given the content of the skyline, the journalists decide to place more weight on European competitions, and so they come up with the following scoring function: $s(o) = o.AO + 2 \cdot o.RG + o.US + 2 \cdot o.WI$. Apply the threshold algorithm to the table above (on the right-hand side) to identify the top-1 player according to the given scoring function. Specify, for each execution round of the algorithm, the threshold and the content of the buffer. No need to indicate all the accesses.

Turn the page to see the other exercises.

C. Physical Databases (9 points)

A retail company records data about customers and receipts of its grocery stores. Table **CUSTOMER** (**CUSTOMERID**, **LastName**, **Name**, **City**, ...) contains 30K tuples over 5K blocks, with a primary hash structure built on **CUSTOMERID** and filling factor below 50%. Table **RECEIPT** (**RECEIPTID**, **Timestamp**, **StoreID**, **CustomerID**, **TotalAmount**) records 10M tuples, primarily stored sequentially ordered by **CustomerID** on 100K blocks. We know that $\text{val}(\text{StoreID})=2\text{K}$ and $\text{val}(\text{City})=10$.

For the following query in the three scenarios below, describe briefly (but precisely) a reasonably efficient query plan and estimate its execution cost, possibly evaluating also alternative plans.

```
SELECT *
FROM CUSTOMER
WHERE City = 'Milan' AND CUSTOMERID IN
      (SELECT CustomerID
       FROM RECEIPT
       WHERE StoreID = 'MI-11' )
```

1. There are no secondary access structures;
2. Table **RECEIPT** has a secondary hash index built on **CustomerID**, with the same hash function used for the primary structure of **CUSTOMER**, with overflow cost 1.3 and 6.5K blocks (including the overflow blocks);
3. Table **RECEIPT** also has a B+ index built on **StoreID** as search key, with $F=90$ and 50K leaf nodes.

D. Trigger (8 points)

A mobile app statistics application manages data stored in a database with the schema reported below. Primary keys are represented with capital letters.

```
COMPANY (COMPANYID, name, address, mail)
APPS (APPID, companyId, tot_installs, tot_uninstalls)
APP_WEEKLY_STATS (APPID, WEEK_NUM, week_installs, week_uninstalls)
ALERT (APPID, WEEK_NUM, delta)
BEST_APPS (APPID, COMPANYID)
```

At the end of each week, an application inserts in table **APP_WEEKLY_STATS** the number of installations and uninstallations of all the apps of all companies occurred in the current week. Some triggers react to the insertion of the weekly statistics of the apps by: a) Updating the total (i.e., cumulative) number of installations and uninstallations of the apps. b) Creating an alert (by inserting a row in the **ALERT** table) if the uninstallations of an app have increased more than 10% with respect to the number of uninstallations observed in the preceding week. The alert reports the number of uninstallations exceeding the number of uninstallations observed the week before (in the **delta** column). c) Updating, only if needed, the **BEST_APP** table recording, for each company, the app with the highest number of total installations.

Hypotheses:

- Weeks of the year are integers from 1 to 52.
- The available apps are fixed and all stored in the **APPS** table. No new apps nor dismissed apps.
- Ignore the case in which two or more apps of the same company have exactly the same number of installations.

Specify any assumption on the database that you may have added to make your triggers work.

Turn the page to see the other exercises.